<

Cours Modélisation et vérification des systèmes informatiques
Exercices
Modélisation d'algorithmes en PlusCal (I)
par Dominique Méry
16 octobre 2026

Exercice 1 *appex4...*
Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA contenant une expressiond e l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence d'erreurs à l'exécution.

Question 1.1 *appex4_1_1.tla*

$\ell_1 : x = 10 \ \wedge \ y = z+x \ \wedge z = 2 \cdot x$
$y := z+x$
$\ell_2 : x = 10 \ \wedge \ y = x+2 \cdot 10$

◇– **Solution de l'exercice 1**

Listing 1 – Module TLA+

```
--------- MODULE appex4_1_1 --------
EXTENDS Integers,TLC
-----------------------------------------------------------
CONSTANTS x0,y0,z0
-----------------------------------------------------------
pre(a,b,c)  == a=10 /\ c=2*a /\ b=c+a
-----------------------------------------------------------
(* Verifying the existence of a triple that satisfies pre(x0;y0,z0) *)
ASSUME pre(x0,y0,z0)
-----------------------------------------------------------
(*
--algorithm  test  {
variables x=x0,z=z0,y=y0,t;
{
l1: assert x=10 /\ z=2*x /\ y = z + x;
y:=z+x;
l2: assert x = 10 /\ y = x + 2*10;
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "6b4ae46f" /\ chksum(tla) = "d33b1a21")
CONSTANT defaultInitValue
VARIABLES x, z, y, t, pc

vars == << x, z, y, t, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ z = z0
        /\ y = y0
        /\ t = defaultInitValue
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=10 /\ z=2*x /\ y = z + x,
                "Failure␣of␣assertion␣at␣line␣15,␣column␣5.")
      /\ y' = z+x
```

```
      /\ pc' = "l2"
      /\ UNCHANGED << x, z, t >>

l2 == /\ pc = "l2"
      /\ Assert(x = 10 /\ y = x + 2*10,
                "Failure of assertion at line 17, column 5.")
      /\ pc' = "Done"
      /\ UNCHANGED << x, z, y, t >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

Next == l1 \/ l2
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

\* ASSUME pre
-----------------------------------------------------------------
(* premiminaries for checking  on  pssible values *)
MAX == 32767  (* 16 bits *)
D == -32768..32767
DD(X) ==  (X # defaultInitValue) => (X \in D)
-----------------------------------------------------------------
Inv ==
    /\ pc \in {"l1","l2","Done"}
    /\ x \in Int /\ y \in Int /\ z \in Int
    /\ pc="l1" =>  x=10 /\  z=2*x /\ y=z+x
    /\ pc="l2" =>   x=10 /\ y=x+2*10
    /\ pc="Done" =>   x=10 /\ y=x+2*10
Safety_Partial_Correctness == pc="Done" =>   x=10 /\ y=x+2*10
Safety_rte ==  DD(x)  /\ DD(y) /\  DD(z) /\ DD(t)
check == Inv /\ Safety_Partial_Correctness /\ Safety_rte
prop == [] (x=x0)
=============================================
```

Listing 2 – Configuration TLC

```
SPECIFICATION Spec
CONSTANT defaultInitValue = defaultInitValue
\* Add statements after this line.
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
    greeting = "Hello"
    x0=10
    y0 =30
     z0 = 20
     defaultInitValue = 50000

PROPERTY
\* Uncomment the previous line and add property names
prop
```

```
INVARIANT
\* Uncomment the previous line and add invariant names
check
```

Fin 1

Question 1.2 *On suppose que p est un nombre premier :*

$\ell_1 : x = 2^p \ \wedge \ y = 2^{p+1} \ \wedge \ x \cdot y = 2^{2 \cdot p+1}$
$x := y+x+2^x$
$\ell_2 : x = 5 \cdot 2^p \ \wedge \ y = 2^{p+1}$

◇– Solution de l'exercice 1

Listing 3 – Module TLA+

```
--------- MODULE appex4_1_2 --------
EXTENDS Integers, Naturals,TLC
CONSTANTS x0,y0,p,maxi
----------------------------------------------------------------
diviseurs(n) == { m \in 1..maxi:  n % m = 0 }
premier(n) ==  diviseurs(n) = {1,n} /\ n # 1
pre == x0=2^p /\ y0=2^(p+1) /\ x0*y0=2^(2*p+1)
----------------------------------------------------------------
ASSUME premier(p) /\ pre
(*

--algorithm  test  {
variables x=x0,y=y0;
{
l1:assert x=2^p /\ y=(2^p)*2 /\  x*y=2^(2*p+1);
x:=y+x+(2^x);
l2:assert x=5*(2^p) /\ y=2^(p+1) ;
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "31a0954b" /\ chksum(tla) = "3211f152")
VARIABLES x, y, pc

vars == << x, y, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ y = y0
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=2^p /\ y=(2^p)*2 /\  x*y=2^(2*p+1),
                "Failure␣of␣assertion␣at␣line␣15,␣column␣4.")
      /\ x' = y+x+(2^x)
      /\ pc' = "l2"
      /\ y' = y

l2 == /\ pc = "l2"
      /\ Assert(x=5*(2^p) /\ y=2^(p+1),
                "Failure␣of␣assertion␣at␣line␣17,␣column␣4.")
      /\ pc' = "Done"
      /\ UNCHANGED << x, y >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

```

```
Next == l1 \/ l2
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

---------------------------------------------------------------
MAX == 32768  (* 16 bits *)
D == 0..32768
DD(X) == ( X \in D)
DDP(X) ==  (X # defaultInitValue) => (X \in D)
Safety_absence ==  DD(x)  /\ DD(y)
Inv ==
    /\ pc \in {"l1","l2","Done"}
    /\ x \in Int /\ y \in Int
    /\ pc="l1" => x=2^p /\ y=2^p*2 /\  x*y=2^(2*p+1)
    /\ pc="l2" =>    x=5*2^p /\ y=2^(p+1)

=============================================
```

Listing 4 – Configuration TLC

```
SPECIFICATION Spec
\* Add statements after this line.
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's declared
\* in the specification file. Comment INIT / NEXT parameters if you use SPECIFICATION.

CONSTANTS
    greeting = "Hello"
    p = 2
    x0 = 4
    y0 = 8
    maxi  = 89

INIT Init
NEXT Next

\* PROPERTY
\* Uncomment the previous line and add property names

\* INVARIANT
\* Uncomment the previous line and add invariant names
```

Fin 1

Question 1.3

$\ell_1 : x = 1 \ \wedge \ y = 12$
$x := 2 \cdot y$
$\ell_2 : x = 1 \ \wedge \ y = 24$

◇– Solution de l'exercice 1

Listing 5 – Module TLA+

```
--------- MODULE appex4_1_3 --------
```

```
EXTENDS Integers,TLC
----------------------------------------------------------------
CONSTANTS x0,y0
----------------------------------------------------------------
pre == x0=1 /\ y0=12

----------------------------------------------------------------
(*

--algorithm  test  {
variables x=x0,y=y0;
{
l1:assert x=1 /\ y=12;
x:=2*y;
l2:assert x=24 /\ y=12;
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "fdb997c5" /\ chksum(tla) = "97306b1b")
VARIABLES x, y, pc

vars == << x, y, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ y = y0
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=1 /\ y=12, "Failure␣of␣assertion␣at␣line␣14,␣column␣4.")
      /\ x' = 2*y
      /\ pc' = "l2"
      /\ y' = y

l2 == /\ pc = "l2"
      /\ Assert(x=24 /\ y=12, "Failure␣of␣assertion␣at␣line␣16,␣column␣4.")
      /\ pc' = "Done"
      /\ UNCHANGED << x, y >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

Next == l1 \/ l2
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

----------------------------------------------------------------
MAX == 32768  (* 16 bits *)
D == 0..32768
DD(X) == ( X \in D)
DDP(X) ==  (X # defaultInitValue) => (X \in D)
----------------------------------------------------------------
Safety_absence ==  DD(x)  /\ DD(y)
Inv ==
    /\ pc \in {"l1","l2","Done"}
    /\ x \in Int /\ y \in Int
```

```
    /\ pc="l1" =>  x=1 /\ y=12
    /\ pc="l2" =>  x=1 /\ y=24

=============================================
```

Listing 6 – Configuration TLC

```
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
    greeting = "Hello"

INIT Init
NEXT Next

\* PROPERTY
\* Uncomment the previous line and add property names

\* INVARIANT
\* Uncomment the previous line and add invariant names
```

Fin 1

Question 1.4

$\ell_1 : x = 11 \ \wedge \ y = 13$
$z := x; x := y; y := z;$
$\ell_2 : x = 26/2 \ \wedge \ y = 33/3$

◇– Solution de l'exercice 1

Listing 7 – Module TLA+

```
--------- MODULE appex4_1_4 --------
EXTENDS Integers,TLC
-----------------------------------------------------------------

(*
--algorithm  test  {
variables x=11,y=13,z;
{
l1:assert x=11 /\ y=13;
z:=x;x:=y;y:=z;
l2:assert x=26 \div 2 /\ y=33 \div 3;

l3:print <<x,y>>;
}
}
*)
\* BEGIN TRANSLATION
CONSTANT defaultInitValue
VARIABLES x, y, z, pc

vars == << x, y, z, pc >>

Init == (* Global variables *)
        /\ x = 11
        /\ y = 13
```

```
        /\ z = defaultInitValue
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=11 /\ y=13, "Failure of assertion at line 10, column 4.")
      /\ z' = x
      /\ x' = y
      /\ y' = z'
      /\ pc' = "l2"

l2 == /\ pc = "l2"
      /\ Assert(x=26 \div 2 /\ y=33 \div 3,
                "Failure of assertion at line 12, column 4.")
      /\ pc' = "l3"
      /\ UNCHANGED << x, y, z >>

l3 == /\ pc = "l3"
      /\ PrintT(<<x,y>>)
      /\ pc' = "Done"
      /\ UNCHANGED << x, y, z >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

Next == l1 \/ l2 \/ l3
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

-----------------------------------------------------------------
MAX == 32768  (* 16 bits *)
D == 0..32768
DD(X) == ( X \in D)
DDP(X) ==  (X # defaultInitValue) => (X \in D)
---------------------------------------
Safety_absence ==  DD(x)  /\ DD(y)
Inv ==
    /\ pc \in {"l1","l2","Done"}
    /\ x \in Int /\ y \in Int
    /\ pc="l1" =>  x=11 /\ y=13
    /\ pc="l2" =>  x=26 \div 2 /\ y=33 \div 3

=========
```

Listing 8 – Configuration TLC

```
\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
    greeting = "Hello"

INIT Init
NEXT Next
```

```
\* PROPERTY
\* Uncomment the previous line and add property names

\* INVARIANT
\* Uncomment the previous line and add invariant names
```

Fin 1

Question 1.5 *Dans cette question, on considère l'algorithme correspondant au calcul du maximum de deux nombres.*

Requires : $x_0, y_0 \in \mathbb{N}$
Ensurees : $z_f = max(x_0, y_0)$

$\ell_0 : \{\ldots\}$
if $X < Y$ **then**
  $\ell_1 : \{\ldots\}$
  $Z := Y;$
  $\ell_2 : \{\ldots\}$
else
  $\ell_3 : \{\ldots\}$
  $Z := X;$
  $\ell_4 : \{\ldots\}$
;
$\ell_5 : \{\ldots\}$

Algorithme 1: MAX annotée

◇– **Solution de l'exercice 1**

Listing 9 – Module TLA+

```
---------------------- MODULE appex4_1_5 --------------------------------
EXTENDS Naturals, Integers, TLC

CONSTANTS x0,y0,min,max

ASSUME x0 \in min..max /\ y0 \in min..max


(*
--algorithm commeonveut {
  variables x=x0,
            y = y0,
            z;

{
 l0:assert x=x0/\ y=y0;
 if ( x < y ) {
  l1:  assert x=x0 /\ y=y0 /\ x<y;
  z:=y;
  l2: assert x=x0 /\ y=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y} ;
  }
else
{
```

```
 l3: assert x=x0 /\ y=y0 /\  x >= y ;
 z:= x;
 l4: assert x=x0 /\ y=y0 /\ z=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y};
 };
 l5: assert x=x0 /\ y=y0 /\ z \in  {x,y} /\ x <= z /\ y <= z;
 print "done";
 l6: assert x=x0 /\ y=y0 /\ z \in  {x,y} /\ x <= z /\ y <= z;

}
}
*)
\* BEGIN TRANSLATION
CONSTANT defaultInitValue
VARIABLES x, y, z, pc

vars == << x, y, z, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ y = y0
        /\ z = defaultInitValue
        /\ pc = "l0"

l0 == /\ pc = "l0"
      /\ Assert(x=x0/\ y=y0, "Failure␣of␣assertion␣at␣line␣16,␣column␣5.")
      /\ IF x < y
            THEN /\ pc' = "l1"
            ELSE /\ pc' = "l3"
      /\ UNCHANGED << x, y, z >>

l1 == /\ pc = "l1"
      /\ Assert(x=x0 /\ y=y0 /\ x<y,
                "Failure␣of␣assertion␣at␣line␣18,␣column␣8.")
      /\ z' = y
      /\ pc' = "l2"
      /\ UNCHANGED << x, y >>

l2 == /\ pc = "l2"
      /\ Assert(x=x0 /\ y=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y},
                "Failure␣of␣assertion␣at␣line␣20,␣column␣7.")
      /\ pc' = "l5"
      /\ UNCHANGED << x, y, z >>

l3 == /\ pc = "l3"
      /\ Assert(x=x0 /\ y=y0 /\  x >= y,
                "Failure␣of␣assertion␣at␣line␣24,␣column␣6.")
      /\ z' = x
      /\ pc' = "l4"
      /\ UNCHANGED << x, y >>

l4 == /\ pc = "l4"
      /\ Assert(x=x0 /\ y=y0 /\ z=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y},
                "Failure␣of␣assertion␣at␣line␣26,␣column␣6.")
      /\ pc' = "l5"
      /\ UNCHANGED << x, y, z >>

l5 == /\ pc = "l5"
      /\ Assert(x=x0 /\ y=y0 /\ z \in  {x,y} /\ x <= z /\ y <= z,
                "Failure␣of␣assertion␣at␣line␣28,␣column␣6.")
      /\ PrintT("done")
      /\ pc' = "l6"
```

```
85        /\ UNCHANGED << x, y, z >>
86
87  l6 == /\ pc = "l6"
88        /\ Assert(x=x0 /\ y=y0 /\ z \in  {x,y} /\ x <= z /\ y <= z,
89                  "Failure␣of␣assertion␣at␣line␣30,␣column␣6.")
90        /\ pc' = "Done"
91        /\ UNCHANGED << x, y, z >>
92
93  (* Allow infinite stuttering to prevent deadlock on termination. *)
94  Terminating == pc = "Done" /\ UNCHANGED vars
95
96  Next == l0 \/ l1 \/ l2 \/ l3 \/ l4 \/ l5 \/ l6
97             \/ Terminating
98
99  Spec == Init /\ [][Next]_vars
100
101 Termination == <>(pc = "Done")
102
103 \* END TRANSLATION
104
105
106 maximum(X,Y,Z) ==  Z \in  {X,Y} /\ X <= Z /\ Y <= Z
107
108
109 Inv ==
110     /\ pc \in {"l0","l1","l2","l3","l4","l5","l6","Done"}
111     /\ x \in Int /\ y \in Int /\ (z#defaultInitValue => z \in Int)
112     /\ pc = "l0" => x=x0/\ y=y0
113     /\ pc="l1" => x=x0 /\ y=y0 /\ x<y
114
115 safetyrte == x \in min..max /\ y \in min..max /\ z \in min.max
116
117
118 safetypcv1 == pc="Done" =>  x=x0 /\ y=y0 /\ z \in  {x,y} /\ x <= z /\ y <= z
119
120
121 safetypcv2 == pc="Done" =>  x=x0 /\ y=y0 /\ maximum(x,y,z)
122
123 ===============================================================
124 \* Modification History
125 \* Last modified Tue Oct 04 09:20:27 CEST 2022 by mery
126 \* Created Wed Sep 09 17:02:47 CEST 2015 by mery
```

Listing 10 – Configuration TLC

```
1
2   \* SPECIFICATION
3   \* Uncomment the previous line and provide the specification name if it's
        declared
4   \* in the specification file. Comment INIT / NEXT parameters if you use
        SPECIFICATION.
5
6   CONSTANTS
7       greeting = "Hello"
8
9   x0 = 89
10  y0 = 789
11  min = 0
12  max = 888888
13  defaultInitValue = 99999999
14
```

```
INIT Init
NEXT Next

\* PROPERTY
\* Uncomment the previous line and add property names

\* INVARIANT
\* Uncomment the previous line and add invariant names
```

Fin 1

Exercice 2 *Annoter chaque algorithme et le traduire avec la traduction de PlusCal en TLA$^+$.*

Question 2.1 *pluscal_loop*

Listing 11 – Fragment

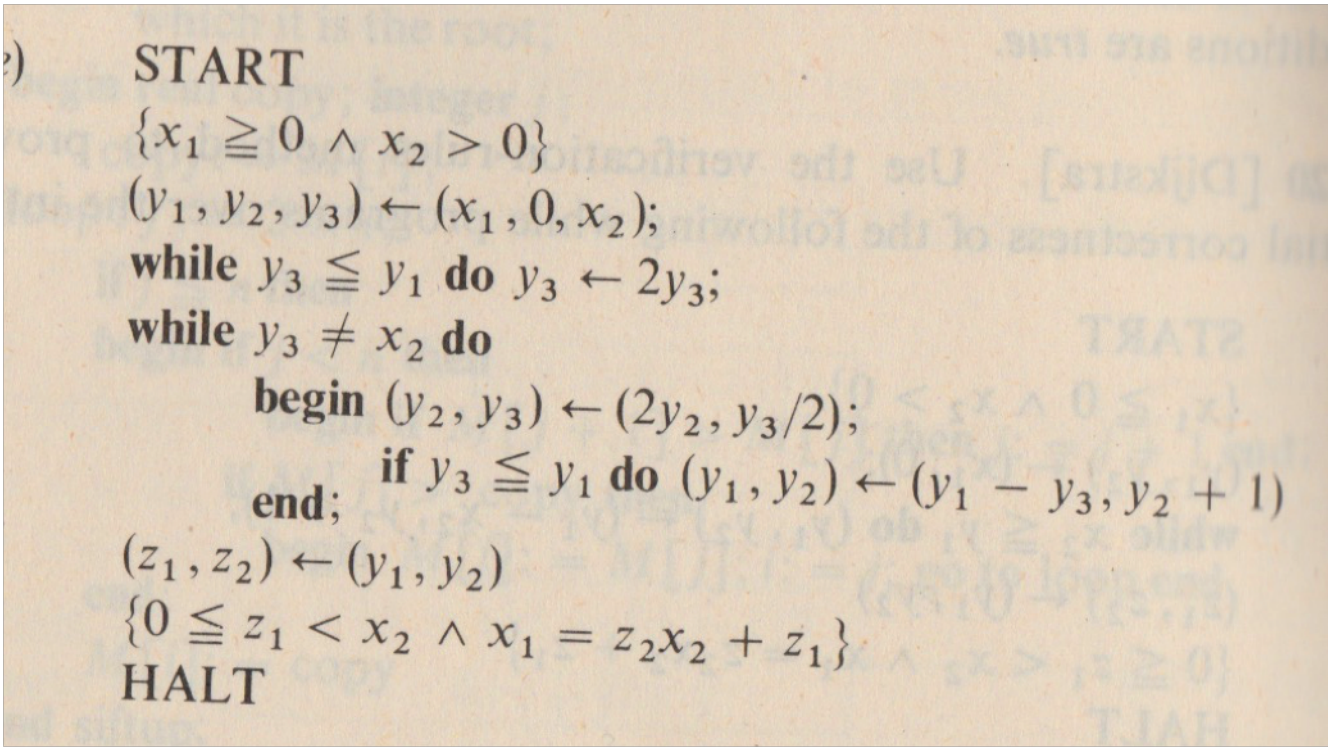
```
          variables x=x0;
{
 while (0<x)
  {
     x := x - 1;
   };
}
}
```

Question 2.2 *pluscal_max*

Listing 12 – FRagment

```
  variables x=a,y = b,z;
{
 if ( x < y ) {
  z:=y;
  }
else
{
 z:= x;
 };
 }
}
```

Exercice 3 *appex4_3.tla*
On considère l'algorithme suivant :



START
$\{x_1 \geqq 0 \wedge x_2 > 0\}$
$(y_1, y_2, y_3) \leftarrow (x_1, 0, x_2)$;
while $y_3 \leqq y_1$ **do** $y_3 \leftarrow 2y_3$;
while $y_3 \neq x_2$ **do**
begin $(y_2, y_3) \leftarrow (2y_2, y_3/2)$;
if $y_3 \leqq y_1$ **do** $(y_1, y_2) \leftarrow (y_1 - y_3, y_2 + 1)$
end;
$(z_1, z_2) \leftarrow (y_1, y_2)$
$\{0 \leqq z_1 < x_2 \wedge x_1 = z_2 x_2 + z_1\}$
HALT

Question 3.1 *Montrer que cet algorithme est aprtiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du langage PlusCal.*

Question 3.2 *Montrer qu'il est sans erreur à l'exécution.*

◇– **Solution de l'exercice 3**

Listing 13 – Module TLA+

```
1  ---------------------- MODULE   appex4_3--------------------------------
2  EXTENDS Naturals, Integers, TLC
3
4  CONSTANT MAXINT,u,v
5
6  (*
7  --algorithm gcdscm {
8    variables x1 =u; (* 1st integer *)
9              x2 = v;  (* 2nd integer *)
10             y1;
11             y2;
12             y3;
13             y4;
14             z1,z2;
15
16
17 {
18      print <<x1, x2>>;
19
20     y1:=x1;
21     y2:=x2;
22     y3:=x2;
23     y4:=0;
24     while (y1 /= y2) {
25
```

```
26        while  ( y1 > y2) {
27          y1:=y1-y2;
28          y4:=y4+y3;
29        };
30        while  ( y2 > y1) {
31          y2:=y2-y1;
32          y3:=y4+y3;
33        };
34
35      };
36      z1 := y1;
37      z2 := y3+y4;
38      print <<x1, x2,z1,z2>>
39  }
40
41  }
42  *)
43  \* BEGIN TRANSLATION
44  CONSTANT defaultInitValue
45  VARIABLES x1, x2, y1, y2, y3, y4, z1, z2, pc
46
47  vars == << x1, x2, y1, y2, y3, y4, z1, z2, pc >>
48
49  Init == (* Global variables *)
50          /\ x1 = u
51          /\ x2 = v
52          /\ y1 = defaultInitValue
53          /\ y2 = defaultInitValue
54          /\ y3 = defaultInitValue
55          /\ y4 = defaultInitValue
56          /\ z1 = defaultInitValue
57          /\ z2 = defaultInitValue
58          /\ pc = "Lbl_1"
59
60  Lbl_1 == /\ pc = "Lbl_1"
61           /\ PrintT(<<x1, x2>>)
62           /\ y1' = x1
63           /\ y2' = x2
64           /\ y3' = x2
65           /\ y4' = 0
66           /\ pc' = "Lbl_2"
67           /\ UNCHANGED << x1, x2, z1, z2 >>
68
69  Lbl_2 == /\ pc = "Lbl_2"
70           /\ IF y1 /= y2
71                 THEN /\ pc' = "Lbl_3"
72                      /\ UNCHANGED << z1, z2 >>
73                 ELSE /\ z1' = y1
74                      /\ z2' = y3+y4
75                      /\ PrintT(<<x1, x2,z1',z2'>>)
76                      /\ pc' = "Done"
77           /\ UNCHANGED << x1, x2, y1, y2, y3, y4 >>
78
79  Lbl_3 == /\ pc = "Lbl_3"
80           /\ IF y1 > y2
81                 THEN /\ y1' = y1-y2
82                      /\ y4' = y4+y3
83                      /\ pc' = "Lbl_3"
84                 ELSE /\ pc' = "Lbl_4"
85                      /\ UNCHANGED << y1, y4 >>
86           /\ UNCHANGED << x1, x2, y2, y3, z1, z2 >>
```

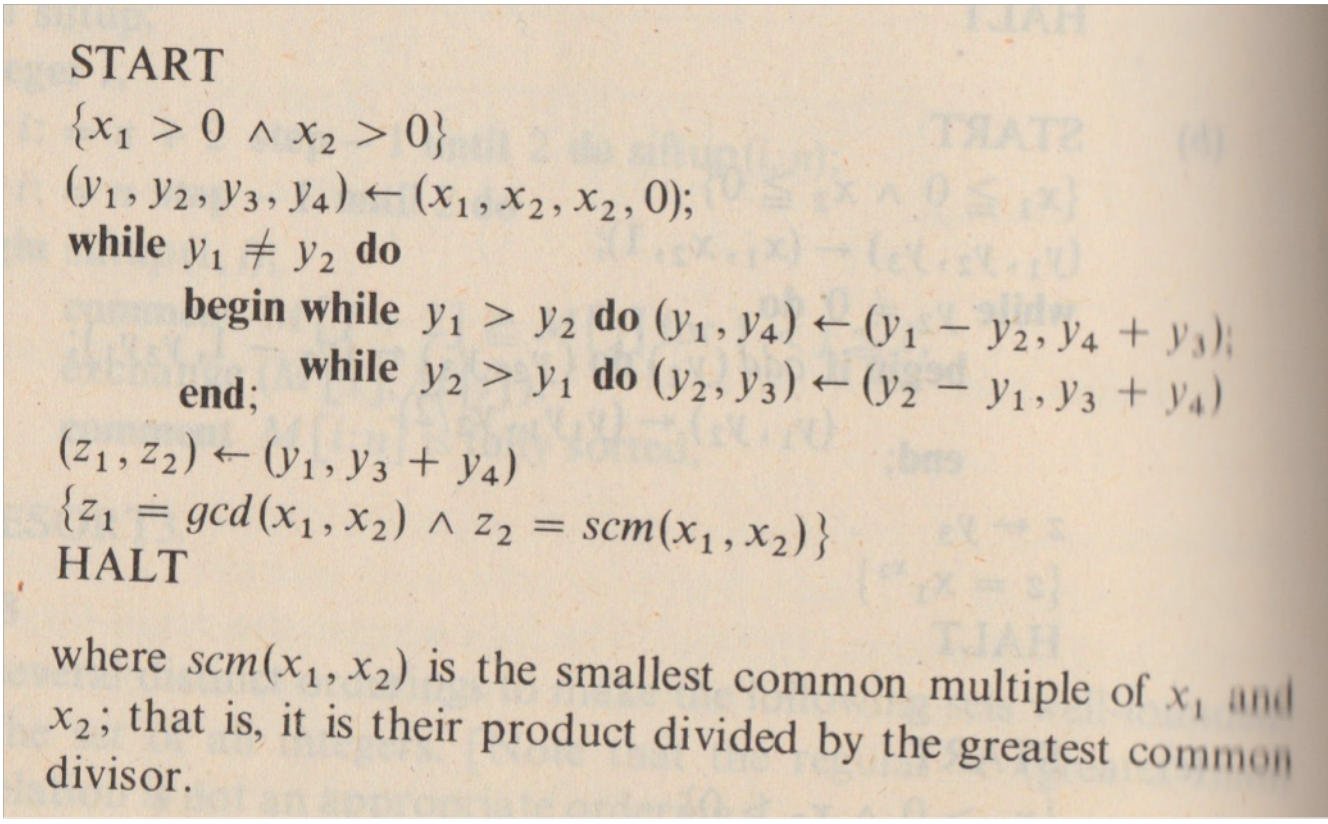
```
87
88  Lbl_4 == /\ pc = "Lbl_4"
89           /\ IF y2 > y1
90                 THEN /\ y2' = y2-y1
91                      /\ y3' = y4+y3
92                      /\ pc' = "Lbl_4"
93                 ELSE /\ pc' = "Lbl_2"
94                      /\ UNCHANGED << y2, y3 >>
95           /\ UNCHANGED << x1, x2, y1, y4, z1, z2 >>
96
97  (* Allow infinite stuttering to prevent deadlock on termination. *)
98  Terminating == pc = "Done" /\ UNCHANGED vars
99
100 Next == Lbl_1 \/ Lbl_2 \/ Lbl_3 \/ Lbl_4
101            \/ Terminating
102
103 Spec == Init /\ [][Next]_vars
104
105 Termination == <>(pc = "Done")
106
107 \* END TRANSLATION
108
109 Q1 == pc # "Done"
110 (* Qpc == pc ="Done" => z1=gcd(x1,x2) /\ z2=scm(x1,x2) *)
111 D == 0..MAXINT\cup {defaultInitValue}
112 Qpc1 == pc = "Done" => ( x1 % z1  = 0 ) /\ (x2 % z1 = 0)
113 Qpc2 == pc = "Done" => ( z2 % x1  = 0 ) /\ (z2 % x2 = 0)
114 Qproperty1 == pc = "Done" => x1*x2=z1*z2
115 Qef == x1 \in D
116 =============================================================================
117 \* Modification History
118 \* Last modified Thu Nov 19 20:42:02 CET 2020 by mery
119 \* Created Wed Sep 09 17:02:47 CEST 2015 by mery
```

Listing 14 – Configuration TLC

```
1  \* SPECIFICATION
2  \* Uncomment the previous line and provide the specification name if it's
       declared
3  \* in the specification file. Comment INIT / NEXT parameters if you use
       SPECIFICATION.
4
5  CONSTANTS
6      greeting = "Hello"
7      MAXINT = 1000
8      u = 67
9      v = 45
10     defaultInitValue = 2000
11
12 INIT Init
13 NEXT Next
14
15 \* PROPERTY
16 \* Uncomment the previous line and add property names
17
18 INVARIANT
19 \* Uncomment the previous line and add invariant names
20 Qpc1
21 Qpc2
22 Qef
```

Fin 3

Exercice 4 *appex4_4.tla*
On considère l'algorithme suivant :


START
$\{x_1 > 0 \wedge x_2 > 0\}$
$(y_1, y_2, y_3, y_4) \leftarrow (x_1, x_2, x_2, 0);$
while $y_1 \neq y_2$ **do**
begin while $y_1 > y_2$ **do** $(y_1, y_4) \leftarrow (y_1 - y_2, y_4 + y_3);$
end; **while** $y_2 > y_1$ **do** $(y_2, y_3) \leftarrow (y_2 - y_1, y_3 + y_4)$
$(z_1, z_2) \leftarrow (y_1, y_3 + y_4)$
$\{z_1 = gcd(x_1, x_2) \wedge z_2 = scm(x_1, x_2)\}$
HALT

where $scm(x_1, x_2)$ is the smallest common multiple of x_1 and x_2; that is, it is their product divided by the greatest common divisor.

Question 4.1 *Montrer que cet algorithme est aprtiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du lanage PlusCal.*

Question 4.2 *Montrer qu'il est sans erreur à l'exécution.*

◇– **Solution de l'exercice 4**

Listing 15 – Module TLA+

```
1  ---------------------- MODULE appex4_5 --------------------------------
2  EXTENDS Naturals, Integers, TLC
3  CONSTANT MAXINT,x10,x20
4
5  (*
6  --algorithm gcdscm {
7    variables x1 =x10; (* 1st integer *)
8              x2 = x20;  (* 2nd integer *)
9              y1;
10             y2;
11             y3;
12             y4;
13             z1,z2;
14
15
```

```
{
     print <<x1, x2>>;
    y1:=x1;
    y2:=x2;
    y3:=x2;
    y4:=0;
    while (y1 /= y2) {

      while  ( y1 > y2) {
        y1:=y1-y2;
        y4:=y4+y3;
      };
      while  ( y2 > y1) {
        y2:=y2-y1;
        y3:=y4+y3;
      };

    };
    z1 := y1;
    z2 := y3+y4;
    print <<x1, x2,z1,z2>>
}

}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "ab42efb3" /\ chksum(tla) = "9f011883")
CONSTANT defaultInitValue
VARIABLES x1, x2, y1, y2, y3, y4, z1, z2, pc

vars == << x1, x2, y1, y2, y3, y4, z1, z2, pc >>

Init == (* Global variables *)
        /\ x1 = x10
        /\ x2 = x20
        /\ y1 = defaultInitValue
        /\ y2 = defaultInitValue
        /\ y3 = defaultInitValue
        /\ y4 = defaultInitValue
        /\ z1 = defaultInitValue
        /\ z2 = defaultInitValue
        /\ pc = "Lbl_1"

Lbl_1 == /\ pc = "Lbl_1"
         /\ PrintT(<<x1, x2>>)
         /\ y1' = x1
         /\ y2' = x2
         /\ y3' = x2
         /\ y4' = 0
         /\ pc' = "Lbl_2"
         /\ UNCHANGED << x1, x2, z1, z2 >>

Lbl_2 == /\ pc = "Lbl_2"
         /\ IF y1 /= y2
               THEN /\ pc' = "Lbl_3"
                    /\ UNCHANGED << z1, z2 >>
               ELSE /\ z1' = y1
                    /\ z2' = y3+y4
                    /\ PrintT(<<x1, x2,z1',z2'>>)
                    /\ pc' = "Done"
         /\ UNCHANGED << x1, x2, y1, y2, y3, y4 >>

```

```
Lbl_3 == /\ pc = "Lbl_3"
         /\ IF y1 > y2
               THEN /\ y1' = y1-y2
                    /\ y4' = y4+y3
                    /\ pc' = "Lbl_3"
               ELSE /\ pc' = "Lbl_4"
                    /\ UNCHANGED << y1, y4 >>
         /\ UNCHANGED << x1, x2, y2, y3, z1, z2 >>

Lbl_4 == /\ pc = "Lbl_4"
         /\ IF y2 > y1
               THEN /\ y2' = y2-y1
                    /\ y3' = y4+y3
                    /\ pc' = "Lbl_4"
               ELSE /\ pc' = "Lbl_2"
                    /\ UNCHANGED << y2, y3 >>
         /\ UNCHANGED << x1, x2, y1, y4, z1, z2 >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

Next == Lbl_1 \/ Lbl_2 \/ Lbl_3 \/ Lbl_4
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION


 Property == pc="Done"=> z1*z2 = x1*x2


Q1 == pc # "Done"
(* Qpc == pc ="Done" => z1=gcd(x1,x2) /\ z2=scm(x1,x2) *)
D == 0..MAXINT\cup {defaultInitValue}
Qpc1 == pc = "Done" => ( x1 % z1  = 0 ) /\ (x2 % z1 = 0)
Qpc2 == pc = "Done" => ( z2 % x1  = 0 ) /\ (z2 % x2 = 0)
Qproperty1 == pc = "Done" => x1*x2=z1*z2
Qef == x1 \in D /\ x2 \in D /\ y1 \in D /\ y3 \in D /\ z1 \in D /\ z2 \in D

=============================================================
\* Modification History
\* Last modified Wed Nov 29 16:59:35 CET 2023 by mery
\* Created Wed Sep 09 17:02:47 CEST 2015 by mery
```

Listing 16 – Configuration TLC

```

\* SPECIFICATION
\* Uncomment the previous line and provide the specification name if it's
    declared
\* in the specification file. Comment INIT / NEXT parameters if you use
    SPECIFICATION.

CONSTANTS
    greeting = "Hello"

x0 = 89
y0 = 789
```

```
11 min = 0
12 max = 888888
13 defaultInitValue = 99999999
14
15 INIT Init
16 NEXT Next
17
18 \* PROPERTY
19 \* Uncomment the previous line and add property names
20
21 \* INVARIANT
22 \* Uncomment the previous line and add invariant names
```

Fin 4